

HIGH LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant using LangGraph & HITL

Further More Information:

- P Jeevan
- Advanced Generative AI Internship
- Innomatics Research Labs

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. System Objectives
4. Scope of the System
5. Overall Architecture
6. Core Components
7. Data Flow Lifecycle
8. Workflow Orchestration using LangGraph
9. Human-in-the-Loop (HITL) Design
10. Technology Stack
11. Scalability Considerations
12. Security & Reliability Considerations
13. Conclusion

1 INTRODUCTION

Write:

Overview

The project implements a Retrieval-Augmented Generation (RAG) based Customer Support Assistant capable of retrieving verified information from PDF knowledge bases and generating context-aware responses using Large Language Models (LLMs).

The system integrates:

- semantic retrieval
- vector embeddings
- workflow orchestration using LangGraph
- conditional routing
- Human-in-the-Loop escalation

to improve response reliability and reduce hallucinations.

2 PROBLEM STATEMENT

Write:

Traditional customer support chatbots suffer from:

- hallucinated responses
- lack of contextual grounding
- inability to handle ambiguous queries
- no escalation mechanism for complex issues

The goal of this system is to create an intelligent support architecture capable of:

- retrieving trusted contextual information
- generating grounded responses
- routing complex queries
- enabling human intervention when required

3 SYSTEM OBJECTIVES

Bullet Points

- Process PDF knowledge bases
- Convert textual data into embeddings
- Store semantic vectors in ChromaDB
- Retrieve relevant contextual information
- Generate grounded responses using LLMs
- Implement graph-based workflow orchestration
- Support conditional routing logic
- Integrate Human-in-the-Loop escalation
- Improve reliability and scalability

4 SCOPE OF THE SYSTEM

In Scope

- PDF ingestion
- Semantic chunking
- Vector search
- Context retrieval
- AI-generated responses
- Workflow orchestration
- Human escalation

Out of Scope

- Multi-language support
- Real-time voice support
- Distributed deployment
- Multi-tenant authentication

5 OVERALL ARCHITECTURE

Architecture Explanation

User Interface Layer

Handles user queries through CLI or web interface.

Document Ingestion Pipeline

Processes uploaded PDFs and extracts textual content.

Chunking Engine

Splits documents into semantically meaningful chunks.

Embedding Layer

Generates vector embeddings representing semantic meaning.

Vector Database (ChromaDB)

Stores embeddings for similarity-based retrieval.

Retrieval Layer

Finds relevant chunks using vector similarity search.

LLM Processing Layer

Combines retrieved context with prompts to generate responses.

LangGraph Workflow Engine

Controls orchestration and state transitions.

Routing Layer

Performs conditional routing based on confidence and query intent.

HITL Module

Escalates uncertain queries to human support agents.

6 CORE COMPONENT DESCRIPTION

Create subsections.

6.1 Document Loader

Responsible for:

- reading PDFs

- extracting text
- preprocessing content

6.2 Chunking Strategy

The document is split into overlapping semantic chunks.

Why?

- improves retrieval precision
- avoids context overflow
- enhances semantic continuity

6.3 Embedding Model

Converts chunks into high-dimensional vectors.

Purpose:

- semantic similarity search
- contextual retrieval

6.4 ChromaDB

Chosen because:

- lightweight
- efficient local vector storage
- easy LangChain integration

6.5 Retriever

Performs:

- top-k similarity search
- semantic matching
- contextual ranking

6.6 LLM Layer

Generates grounded responses using:

- retrieved context
- prompt templates
- user query

6.7 LangGraph Engine

Implements:

- node orchestration
- state transitions

- workflow execution

6.8 Routing Layer

Determines:

- automated response path
- escalation path

based on:

- confidence score
- query complexity
- missing context

6.9 HITL Module

Provides:

- manual intervention
- human validation
- fallback reliability

7 DATA FLOW LIFECYCLE

Write sequential flow

1. PDF uploaded
2. Text extracted
3. Text chunked
4. Embeddings generated
5. Stored in ChromaDB
6. User submits query
7. Query embedding generated
8. Similar chunks retrieved
9. Context passed to LLM
10. Response generated
11. Routing logic evaluated
12. Escalation triggered if needed
13. Final response returned

8 LANGGRAPH WORKFLOW

Explanation

The workflow consists of:

- Input Node
- Processing Node
- Retrieval Node
- Decision Node
- Output Node
- Escalation Node

State Object Includes

- query
- retrieved chunks
- confidence score
- generated response
- escalation status

9 HITL DESIGN

Escalation Conditions

Triggered when:

- low confidence retrieval
- insufficient context
- ambiguous query intent
- high-risk customer issue

Human Workflow

1. Query escalated
2. Human reviews context
3. Human provides response
4. Response returned to user
5. Optional feedback stored

10 TECHNOLOGY STACK

Component	Technology
------------------	-------------------

Programming	Python
-------------	--------

Component	Technology
Framework	LangChain
Workflow Engine	LangGraph
Vector DB	ChromaDB
LLM	OpenAI GPT
Embeddings	OpenAI Embeddings
PDF Loader	PyPDFLoader
Frontend	CLI/Web
Orchestration	LangGraph

11 SCALABILITY CONSIDERATIONS

Discuss

Large Documents

- chunk optimization
- streaming ingestion

Query Load

- caching
- async pipelines

Retrieval Latency

- optimized indexing
- top-k filtering

Future Scale

- distributed vector DB
 - cloud deployment
-

12 SECURITY & RELIABILITY

Mention

- context grounding reduces hallucinations
- HITL improves reliability
- fallback handling
- error detection

- controlled retrieval pipeline
-

13 CONCLUSION

Write

The proposed architecture demonstrates a scalable and reliable AI support system combining:

- Retrieval-Augmented Generation
- workflow orchestration
- semantic retrieval
- conditional decision routing
- human escalation mechanisms

The design prioritizes reliability, modularity, and production-oriented extensibility.